

勉強会 / 2026-06

A2UI 入門

LLMの出力を「文章」から
「操作できるUI」へ

クラウドに一切つながらない端末内LLMデモ — Gemma 4 E2B × flutter_gemma × A2UI v0.9

01 このデモで何を見せるか

① オフライン

機内モードのままLLMが動きます。

プライバシー / 低レイテンシ / オフライン耐性

② A2UI

LLMの出力が"文章"ではなく

操作可能なUIになる

= Agent-to-UI の体験

この2つが同時に成立するのが新しいところです。

02 課題：LLM出力は"文章"止まり

テキストを返すだけでは、ユーザーは結局自分でボタンを探し、フォームを埋めることになります。

→ LLMがUIを記述する プロトコル = **A2UI**

```
// LLMの出力がそのまま画面になる  
{"version":"v0.9","createSurface":{"surfaceId":"edit_style_picker", ...}}
```

03 さらに、クラウド依存をやめたい

プライバシー

データが端末外に出ない

レイテンシ

ネットワーク往復ゼロ

オフライン

圏外でも動く

→ 推論を **端末内の小型LLM** で完結 (Gemma 4 E2B)

04 そもそも A2UI とは？

- ▶ Google 発のオープンプロジェクト **Agent-to-User Interface** (Apache 2.0 · a2ui.org)。エージェントの返事を「文章」ではなく **操作可能なUI** するための共通プロトコル
- ▶ エージェントは UI を **宣言的な JSON** (コンポーネントの木+データモデル) で送ります。実行コードは送しません
- ▶ クライアントは **信頼済みカタログ** にある自前のネイティブウィジェット (Flutter / React / SwiftUI...) にマップして描画 → **任意コード実行なし・UIインジェクションに強い**
- ▶ **JSONL ストリーミング前提** → 生成の途中から描画できる (progressive rendering)

1つの A2UI ペイロードを、各プラットフォームが**自分の部品**で描きます。本デモは v0.9 を Flutter で自前実装しています (パース → 検証 → 描画)。

05 A2UI 以前 / 以後：何が変わったのか

以前：各自が自作（車輪の再発明）

- ▶ LLM に HTML/JSX を直接書かせて描画 → 任意コード実行のリスク・プラットフォーム固定
- ▶ 「コンポーネント＝ツール呼び出し」方式 → 画面ごとにツール定義を自作、組み合わせの自由度が低い
- ▶ 各社独自の Server-Driven UI 規格 → エージェント側もクライアント側も毎回作り直し

以後：プロトコルとして規格化

- ▶ エージェントは**1つの形式**を学べば、どのクライアントにも画面を出せます
- ▶ クライアントは**レンダラーを1回**書けば、任意のエージェントと繋がります
- ▶ 安全モデルが規格に内蔵（データでありコードではない＋信頼カタログ）
- ▶ レンダラーやツールを**エコシステムで共有**できます

MCP がエージェント⇔ツールを規格化したのと同じ構図で、A2UI はエージェント⇔画面を規格化しました。

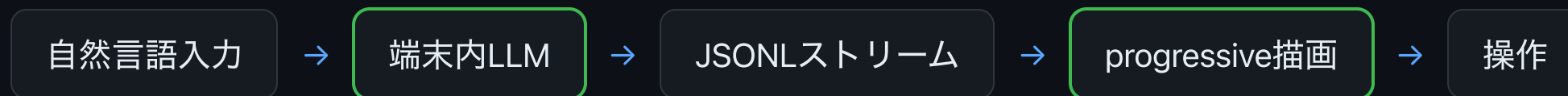
役割分担: A2A＝エージェント間 / AG-UI＝フロントとの通信路 / A2UI＝流れる「UIの中身」

06 A2UI のエコシステムと参考資料

仕様・実装	github.com/a2ui-project/a2ui / a2ui.org (quickstart あり・Apache 2.0)
ステータス	v0.9.1 (Public Preview) 、v1.0 Candidate が進行中
レンダラー	Lit / Angular / React / Web Core、Flutter (GenUI SDK)。Compose・SwiftUI は計画中
転送路	A2A、AG-UI (CopilotKit)。REST は計画中
遊べる場所	a2ui-composer.ag-ui.com — ブラウザ上で A2UI を生成・プレビュー (/theater にレストラン検索などのシナリオデモ)
CopilotKit 連携	Zod でコンポーネントを定義すると説明がプロンプトへ自動注入される「agent-aware components」

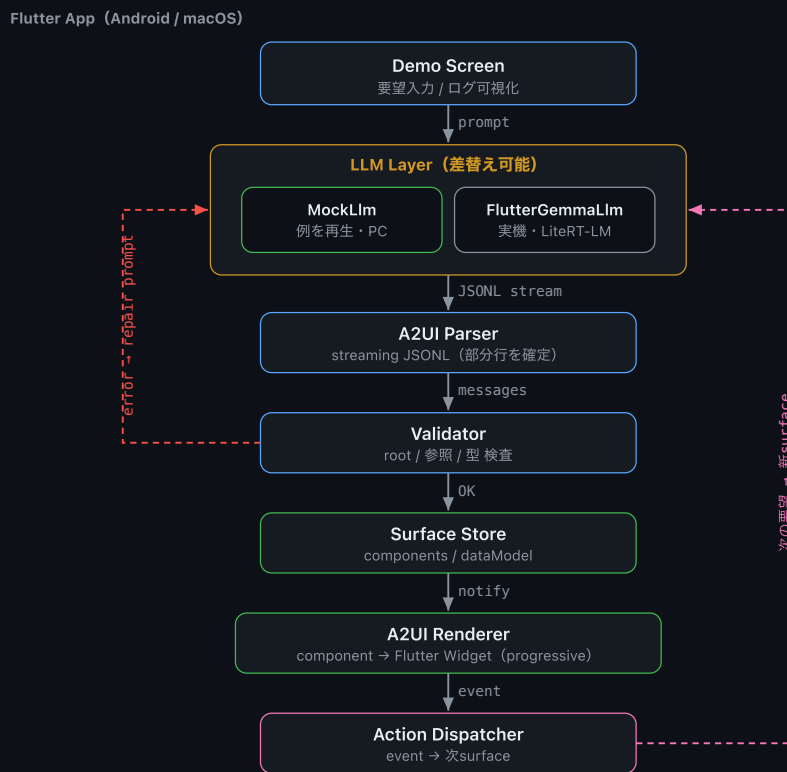
本デモはこのうち **v0.9 のプロトコル部分を Flutter で自前実装**したものです（レンダラー・検証・自己修正は独自）。

07 デモの流れ



「機内モードで」自然言語を入れると、端末内LLMがA2UI JSONを少しずつ出力し、画面が**その場で組み上がって**いきます。

08 全体アーキテクチャ



中心思想：LLM とレンダラーを疎結合に。重い側はインターフェースの裏に隠します。

09 A2UIプロトコル：4種のメッセージ

JSONL（1行1メッセージ）。トップレベルキー1つで判別します。

メッセージ	役割
<code>createSurface</code>	画面を作る（最初に1回）
<code>updateComponents</code>	コンポーネントを置く（root必須・フラット配列）
<code>updateDataModel</code>	表示データを入れる（path = JSON Pointer）
<code>deleteSurface</code>	画面を消す

全メッセージに `"version": "v0.9"` を含めます。

10 A2UI の基本概念 ①：Surface とカタログ

Surface = 画面の単位

```
{"createSurface":{  
  "surfaceId":"s1",  
  "catalogId":"https://.../support"}}}
```

- ▶ createSurface で生まれ、updateComponents で組み上がり、deleteSurface で片付きます
- ▶ エージェントは複数の surface を同時に持てます（画面の差し替え・多段フロー）

カタログ = 信頼の境界

- ▶ surface 生成時に「**使ってよい部品の辞書**」を宣言します（catalogId）
- ▶ カタログ外のコンポーネントは**クライアント側が拒否**します — これが「データであってコードではない」の実体です
- ▶ 標準の basic カタログのほか、**自社デザインシステムを独自カタログ**として公開できます

本実装では Surface / SurfaceStore がライフサイクルを、A2uiValidator がカタログ外の拒否（UNKNOWN_COMPONENT）を担当します。

11 A2UI の基本概念 ②：データモデルとイベント

データモデル = 画面の「値」置き場

```
// コンポーネントは「住所」だけ持つ
"value":{"path":"/reply/draft"}
// 値は別メッセージで届く
{"updateDataModel":{"path":"/reply/draft",
  "value":"お待たせしております..."}}
```

- ▶ 構造とデータを分離 → 値だけ後から差し替えられます

イベント = 操作がエージェントへ還る

- ▶ Button や QuickActions のタップは **event** としてアプリ／エージェントに通知されます
- ▶ TextField 等の入力は dataModel に書き戻ります (双方向バインド)
- ▶ つまり「読むだけの返事」ではなく、操作の結果で次の生成が走るループが閉じます

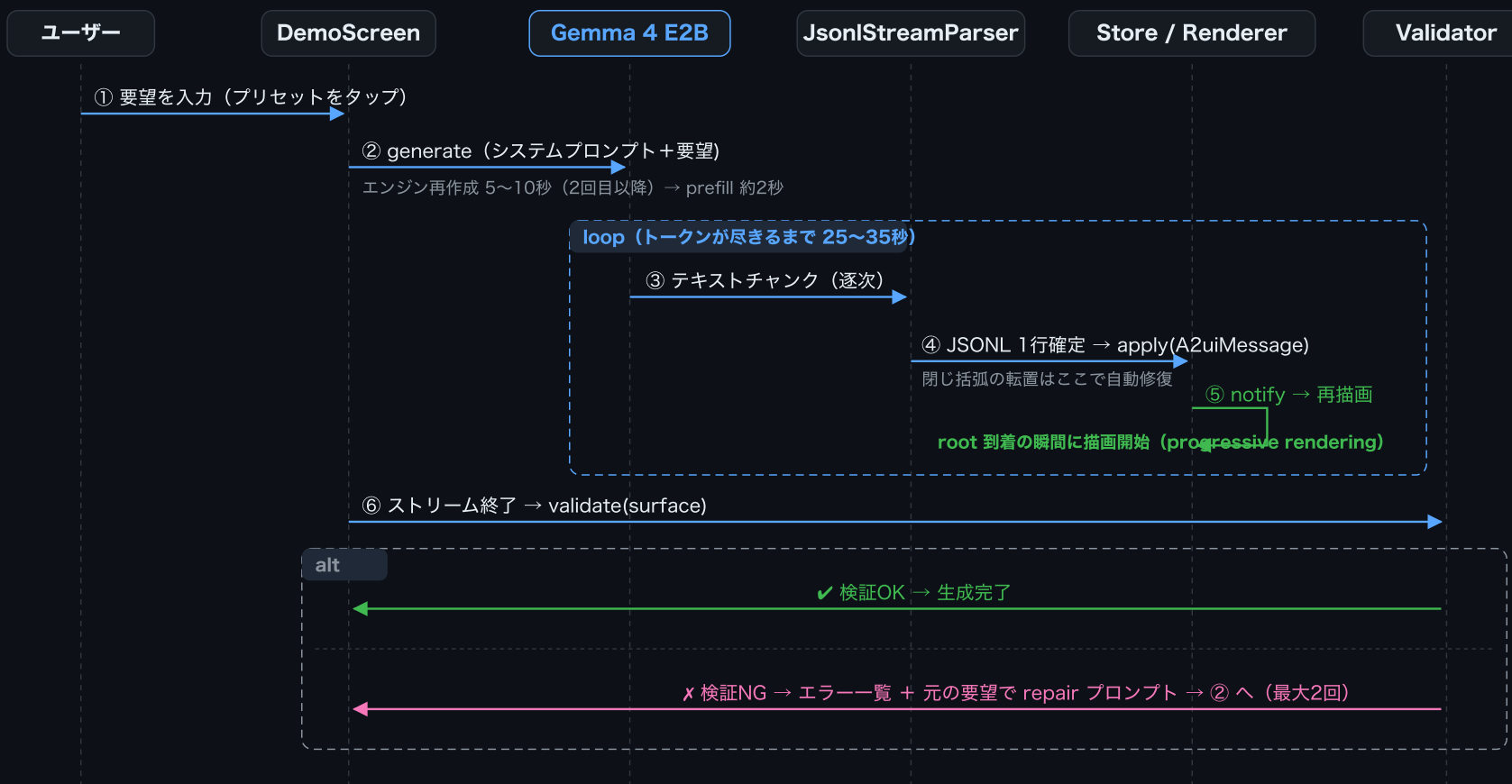
本実装では DataModel / DataBinding と onEvent が担当。タップ作文の compose: イベントが「操作 → 次の画面を生成」の実例です。

12 A2UI の概念 ⇔ 本実装の対応図

A2UI が提唱する概念	本実装のクラス	ファイル
4種のメッセージ（createSurface / updateComponents / updateDataModel / deleteSurface）	A2uiMessage（sealed） + CreateSurface 等4クラス	core/a2ui/message/
コンポーネント（id + 種類 + props のフラット配列）	A2uiComponent	core/a2ui/component/
JSONL ストリーミング	JsonlStreamParser（+閉じ括弧の自動修復）	core/a2ui/parser/
Surface（画面の単位）と状態	Surface / SurfaceStore	core/a2ui/state/
データモデル × JSON Pointer（双方向バインド）	DataModel / DataBinding	state/・ui/renderer/
信頼済みカタログ	supportCatalogId + カタログ定義、A2uiValidator がカタログ外を拒否	llm/prompt/・validation/
ネイティブウィジェットへのマップ	A2uiRenderer（component → Flutter Widget）	ui/renderer/
（本実装の独自拡張）検証 → 自己修正	A2uiValidator + buildRepairPrompt + 暴走ガード	validation/・llm/prompt/

仕様の概念がほぼ1対1でクラスに対応しています。最下段の自己修正だけは仕様外で、**小型LLM向けに本実装が足した部分**です。

13 シーケンス：要望から描画・自己修正まで



描画後のタップは onEvent → dataModel へ書き戻し（双方向バインド）。compose: 系イベントなら ① 相当として次の画面を生成します。

14 構造：フラットな配列が木になり、画面になる

① LLM が吐くのはフラットな配列（請求デモの実物）

```
[{"id": "root", "component": "Column", "children": ["h", "r"]},  
 {"id": "h", "component": "InquiryHeader",  
  "customer": "田中", "priority": "high"},  
 {"id": "r", "component": "ReplyBox",  
  "value": {"path": "/reply/draft"}}
```

ネストなし=1行ずつ独立して届く（ストリーミング向き・小型LLMでも壊れにくい）

② renderer が id を辿って木に → Flutter へ

```
root Column      → Column(children:[  
  | h InquiryHeader → InquiryHeader(田中, 高),  
  | r ReplyBox     → TextField(...),  
  ])
```

③ 値は JSON Pointer で「あとから」入る

```
// 別メッセージで実データが届く  
updateDataModel {"path": "/reply/draft",  
  "value": "お待たせしております。..."}  
// → ReplyBox の TextField に文章が入る  
// ユーザーが編集すると dataModel に書き戻る（双方向）
```

UIの**構造**と**データ**を分離 → 構造はそのまま、updateDataModel だけで画面の値が差し替わります。

15 コンポーネント → Flutter (basic 8種)

A2UI	Flutter	備考
Text	Text / MarkdownBody	variant→TextStyle
Image	Image.network	フォールバック付き
Column/Row	Column / Row	children を id解決して再帰
Card	Card	単一子を囲む
Button	FilledButton等	押下で event 発火
TextField	TextField	dataModelへ双方向
ChoicePicker	SegmentedButton	選択値を双方向

16 見せ場：progressive rendering

小型モデルは生成が遅いです。

それを弱みではなく見せ場に変えます。

```
token stream → [行バッファ] → 改行で確定 → parse → message
createSurface      → 空の画面
updateComponents(root) → 画面が"出る" ← ここで描画開始
updateComponents(...) → 追加分を埋める
updateDataModel    → 文言・値が"入る"
```

rootさえ来れば描画が始まります。遅い端末ほど progressive が活きます。

17 バリデーション & 自己修正ループ



失敗	自己修正での指示
rootが無い	id=rootを必ず1つ含める
未定義idを参照	childrenのidは必ず定義する
カタログ外の型	8種以外は禁止
コードフェンス混入	JSON以外を出力しない

リトライ上限2回で打止め → エラーUI

18 LLMレイヤ（差替え可能）

```
abstract class LlmBackend {  
    Stream<String> generate({required system, required user});  
    Future<void> warmup();    // 初回ロードを前倒し  
}
```

MockLlm

台本のJSONLを再生します。

決定的・どの端末でも動く検証用バックエンド

FlutterGemmaLlm

実機推論。temp 0.2 / topK 1 の**低ランダム設定**
+ 検証→自己修正で構文を担保します

採用モデル：Gemma 4 E2B（LiteRT-LM・約2.4GB・認証不要DL・端末内に使い回し）

19 オンデバイスLLMの選定

モデル	サイズ (4bit)	評価	判断
Qwen2.5 0.5B/1.5B	約0.4～ 1GB	最軽量・最速。ただし厳密なJSON構造と日本語UI文言に不安、flutter_gemma 経路の実績も薄い	候補止 まり
Gemma 3n E2B	約3GB	最初に統合した世代。動くが、配布が gated で当時の MediaPipe .task 経路	→ 移行
Gemma 4 E2B	約 2.4GB	実効2Bの MatFormer。LiteRT-LM + GPU で Pixel 6a 実測 25～35秒/画面。few- shot 1例で valid な A2UI。認証不要でDL可	✓ 採用
Gemma 4 E4B	約4.4GB	品質は一段上だが、Pixel 6a（RAM 6GB）ではロードと生成速度が厳しい	見送り

選定軸: ① 壊れないJSON（構造生成の安定）② **Pixel 6a で GPU 推論が回るサイズ** ③ 認
証不要で配布できる ④ flutter_gemma / LiteRT-LM 対応。

採用モデルの配布元: huggingface.co/litert-community/gemma-4-E2B-it-litert-lm（公式 LiteRT
Community・ログイン/トークン不要で直DL可）

20 E2B の「E」 = Effective (実効)

	総パラメータ	実効パラメータ	実行時メモリ目安
Gemma 4 E2B	約5B	約2B	約2GB～
Gemma 4 E4B	約8B	約4B	約3GB～

MatFormer (マトリョーシカ構造)

大きいモデルの中に小さいモデルが**入れ子で同時学習**されます。E2B は E4B の中に最適化された**サブモデル**です。

Per-Layer Embeddings (PLE)

各層がトークンごとの埋め込み表を持つ。表は大きい**が参照するだけ**なので、GPU の高速メモリに**常駐させなくてよい**=実効に数えません。

= 「**5B級の品質の一部を、2B並みのメモリ・速度で**」動かすオンデバイス特化設計です
(命名は Gemma 3n 世代から踏襲)。

21 実装マイルストーン

#	内容	LLM
M1	モデル+パーサ+レンダラー+MockLlm（デスクトップ描画）	不要
M2	バリデーション+自己修正+双方向バインド	不要
M3	flutter_gemma統合（Android実機・機内モード）	実機
M4	独自カタログ1種（発展）	—

M1・M2はLLM不要で全テスト可能 → LLMの不確実性とUI品質の検証を分離できます。

22 主要リスクと対策

- ▶ 実機モデルのロード遅延 → `warmup()`を起動時に / 小型量子化版
- ▶ 小型モデルがJSONを壊す → `temp`低・`topK=1` + パーサ堅牢化 + 検証→自己修正
- ▶ `children`循環参照・深いネスト → `visited`集合 + 深さ上限ガード
- ! 実機・モデルの不調 → LLM層を差替え可能に (Mock ↔ Gemma をワンタップ切替)

23 実際に遭遇した罠：2回目の生成から壊れる

- ▶ **症状**：エンジン作成後の最初の生成だけ正常。
2回目以降は最初のトークンから縮退ループ (o o o.../bolbolbol...)
- ▶ **切り分け**：1回目に成功した同じプロンプトが4回目では暴走 → 内容ではなく実行順序に依存と特定 (topK=1 で毎回再現)
- ▶ **原因**：LiteRT-LM のセッション状態汚染。
flutter_gemma 0.16.5 の KV-cache bleed 修正 (#308) でも完治せず
- ! **対処**：生成ごとにエンジンを再作成して常に「1回目」の状態で推論 (+5~10秒)

暴走ガード（2000字打ち切り）+生出力ログで被害を限定して調査しました。

A2UI ストリーム ログ

▶ 生成開始 (端末内LLM: Gemma 4 E2B)

[illegible]

A2UI ストリーム ログ

▶ 生成開始 (端末内LLM: Gemma 4 E2B)

⚠ 有効なメッセージが 2000 文字以上出ないため打ち切り
x NO SURFACE サーフェスが生成されませんでした

実際の暴走ログ：非JSON行のスキップが延々と続き（上）、ガードが打ち切って
NO_SURFACE（下）

24 次のステップ：独自カタログ

basic カタログを **自社デザインシステム** に差し替えます。

EditStyleCard

編集スタイルをサムネ付き
で選ぶ

VideoPreviewTile

生成中/完成プレビュー

ShareScopeSelector

公開範囲セレクト

`catalogId`を独自URIにするだけ → LLMが**自社の語彙**でUIを書きます。

25 実務で入れるなら：いまの懸念点

懸念	いまの状態	解決されると...
仕様の若さ	v0.9 (Public Preview)。v0.8→0.9 で形式変更の実績あり	v1.0 安定化で本番採用の土台に
カタログの標準化	basic は8種で実務UIには不足。独自カタログは相互運用性とトレードオフ	業界共通カタログ+デザインシステム連携が出れば一気に楽に
品質保証	生成UIは事実上無限パターン。スナップショットテストや a11y 検証の定石が未整備	検証層・テストツールの整備で QA が回るように
確定性との使い分け	決済・法的表示など「必ずこのUI」が要る画面には不向き	静的UIとの併用パターン（ハイブリッド）が定石化
プラットフォーム対応	SwiftUI / Compose レンダラーは計画中。ツールも発展途上	全主要プラットフォームが揃えば「書けば全部に出る」が現実 に

逆に言えば、**埋まりつつある穴ばかり**です。MCP も同じ道（仕様安定 → エコシステム拡大 → 一気に普及）を辿りました。

「触るなら、波が来る前のいま」が今日の持ち帰りです。

26 嬉しいUXの正体：4つの「～ない」

体験	中身	どこから来るか
探さない	次にやる操作が"もう並んでいる"	A2UI×LLM — 文脈から都度生成
打ち込まない	文章でなく"操作できるUI"が出る	A2UI — 双方向バインド
待たされない	root到着で即描画（組み上がる過程が見える）	A2UI×端末内 — JSONLストリーム＋ローカル即時
送られない	私的データを読んでも端末内で完結	端末内LLM — データが外に出ない

＝「自分を分かってくれて、主導権はこちらにある」という感触です。A2UI 単体でもオンデバイス単体でもなく、掛け算で生まれます。

27 reactive → proactive、そして見たい未来



だんだん"勝手にやってくれる"度が上がっていきます。そして、ここまでの全てが**オフライン・端末内**で動いています。

UIは「作る」から「その場で生まれる」へ。

28 改めて：どこが LLM（動的）か

静的（事前実装・固定）

- カタログ＝コンポーネント定義
InquiryHeader / ReplyBox / ...
 - レンダラー（component → Widget）
 - パーサ / Validator / データモデル
 - システムプロンプト
- = 部品と「組み立て工場」

動的（LLMが実行時に生成）

- どのコンポーネントを選ぶか
 - どう並べ・入れ子にするか
= surface の構造
 - 文言・データ
件名 / 会話 / 返信ドラフト
- = 「献立と盛り付け」だけ。Dartもピクセルも書かない

```
// 同じ1行に静的と動的が同居している
{"component":"InquiryHeader",           ← 型はカタログ語彙（静的）
 "subject":"請求額が二重","priority":"high"} ← 中身はLLM（動的）
```

LLM が書くのは **JSONの設計図** だけです。実装（部品）はすべてアプリ側にあります。

29 実機デモ (Pixel 6a・オフライン)



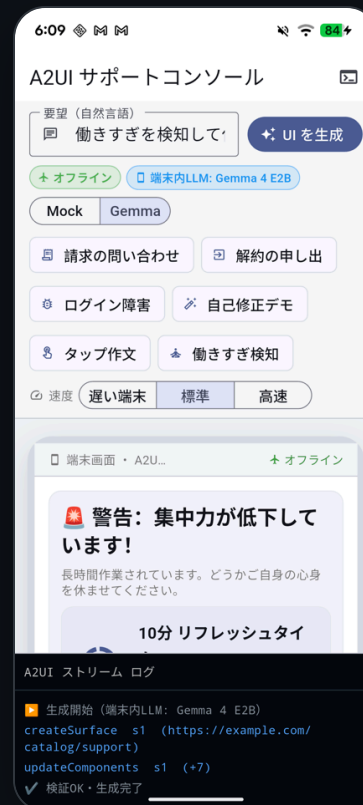
機内モード相当・端末内LLM (Gemma 4 E2B ↔ Mock 切替)。要望ごとに違う画面がその場で生成されます。3枚とも本日 Pixel 6a 実機 Gemma で生成・検証OKの実キャプチャです。

実測 (Pixel 6a・GPU・オフライン) : 初回トークン 約2秒 / 1画面 25~35秒 / 連続生成はエンジン再初期化 +5~10秒

30 本命デモ：打ち込まない × proactive



タップ作文 → テキスト + 画像を一斉生成 (Mock 台本)



働きすぎ検知 → 仮眠タイマーが向こうから (実機 Gemma)

左：キーボード未使用、タップだけで返信 + 画像が組み上がります。右：働きすぎを察知してUIが**向こうから**現れます (Pixel 6a 実機 Gemma の実キャプチャ)。

まとめ

端末内LLMが UIを生成する世界

- ✓ オフライン × A2UI が同時に成立
- ✓ 4種メッセージ / 隣接リスト / JSON Pointer / 自己修正
- ✓ progressive rendering で"遅さ"を体験に変える
- ✓ 独自カタログで自社デザインシステムをAIに組みせる

